

ISM3232 — Week 13 Lab

Capstone Design & SQL Foundations — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 13 · Unit 4 · Capstone Build

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Timing Plan (75 minutes)	3
6 Segment-by-Segment Walkthrough	4
6.1 Intro (0:00–0:04)	4
6.2 Part 1 — Capstone Project Proposal (0:04–0:26, 22 min)	4
6.3 Part 2 — SQL Fundamentals (0:26–0:42, 16 min)	9
6.4 Part 3 — Design Your Capstone Schema (0:42–0:56, 14 min)	13
6.5 Part 4 — <code>sqlite3</code> Shell Reference (0:56–1:04, 8 min)	16
6.6 Part 5 — Ritual and Push (1:04–1:10, 6 min)	18
7 Wrap-Up (last ~5 minutes)	19
8 Appendix A — Full Worked Example (PROPOSAL.md + schema.sql + SQL Reference)	19
9 Appendix B — Extra Practice (only if the class finishes early)	21

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 13 Lab — Capstone Design & SQL Foundations
Unit	Unit 4 · Capstone Build
Class length	Full class period (75 minutes)
Format	Independent proposal writing (instructor-gated) + guided SQL shell practice
Prerequisites	Weeks 10–12: full OOP fluency — classes, composition, independent design and build
Student-facing lab page	Week 13 In-Class Lab — Module 7A & 7B, “Project Proposal and SQL Foundations”
Parts covered	Part 1 (capstone proposal) – Part 5 (ritual + push)
Submission	PROPOSAL.md (instructor-approved) + schema.sql + screenshot, GitHub URL, Canvas

This is the highest-stakes single lab of the semester so far, for one specific reason: **the capstone proposal written today must be reviewed and approved before any capstone code is written in Week 14, and the entire remaining four weeks of the course build on it.** The lab page’s own warning deserves to be taken at face value: a vague proposal produces an undefined system that is genuinely impossible to finish in four weeks. This lab also introduces SQL for the first time — real, hands-on `sqlite3` shell work — which pairs directly with Week 11’s OOP-to-SQL mapping table and gives that abstract exercise concrete, executable form.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Write a complete, specific capstone proposal — business problem, primary user, records stored, business rules, planned classes, tables, Streamlit features, and a GenAI feature — concrete enough to build from.
2. Create a table with `CREATE TABLE`, specifying column types and constraints (`NOT NULL`, `DEFAULT`, `PRIMARY KEY AUTOINCREMENT`).
3. Insert, query, filter, update, and aggregate data using `INSERT`, `SELECT`, `WHERE`, `UPDATE ... SET`, and `GROUP BY` with `COUNT/SUM`.
4. Design and test a `schema.sql` for their own capstone system in the `sqlite3` shell.

3 Before Class — Setup Checklist

- ☐ **This is a two-gate lab, and both gates matter — plan review-queue logistics for both, not just Part 1.** The capstone proposal (Part 1) needs your sign-off before Week 14; unlike Week 12's same-day design gate, this approval genuinely blocks a student's next *class session*, not just their next hour — treat a rushed or skipped review here as a real problem to avoid, not a minor scheduling inconvenience.
- ☐ Rehearse the full Part 2 `sqlite3` shell sequence yourself before class, including the `.mode column / .headers` on formatting commands — these are easy to forget and their absence makes query output substantially harder to read live on a projector.
- ☐ Prepare your own capstone-adjacent `schema.sql` example (this guide continues the Week 12 event-registration domain) to demonstrate Part 3's process, distinct from any specific student's actual proposed system.
- ☐ Confirm `sqlite3` is installed and runs from the terminal on a few sampled student machines before class — it ships with macOS and most Linux distributions by default, but Windows/WSL setups may need an explicit install step per your course's precourse setup page.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, `sqlite3` (command-line shell)
- Students: a fresh `~/ism3232/module07_final_project/` folder — the capstone's home for the rest of the semester

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: “this proposal is what Weeks 14–16 build”	4
0:04–0:26	Part 1 — Capstone project proposal (write + review)	22
0:26–0:42	Part 2 — SQL fundamentals in the <code>sqlite3</code> shell	16
0:42–0:56	Part 3 — Design your capstone schema	14
0:56–1:04	Part 4 — <code>sqlite3</code> shell reference (README)	8
1:04–1:10	Part 5 — Ritual and push	6
1:10–1:15	Wrap-up, submission checklist	5

Part 1 receives the most protected time of the whole lab, for the same structural reason as Week 12's design gate, amplified: the stakes of an under-baked proposal are higher here, since it governs four remaining weeks, not one.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Everything you build for the rest of this semester — Weeks 14 through 16 — is defined by what you write today. Not a rough idea: a specific, ten-field proposal that I have to approve before you write a single line of capstone code next week. Vague answers today become an undefined, unfinishable project in three weeks. Take this seriously, and ask questions now, while there’s still time to adjust.”

Do: Write on the board: **Proposal approved by instructor → Week 14 code begins. No sign-off, no green light.**

6.2 Part 1 — Capstone Project Proposal (0:04–0:26, 22 min)

Teaching goal: A complete, specific, ten-field proposal — the single most consequential document of the semester’s second half.

Say to the class:

“Ten fields. I want specific answers, not aspirational ones — ‘a system for managing things’ is not a business problem; ‘the office manager who currently tracks 40 travel requests a month in a spreadsheet’ is. Every vague answer today is a decision you’ll be forced to make anyway, later, under time pressure — make it now, deliberately.”

Do:

```
cd ~/ism3232/module07_final_project
python3 -m venv .venv && source .venv/bin/activate
pip install pytest ruff && pip freeze > requirements.txt
echo '.venv/' > .gitignore && echo '__pycache__/' >> .gitignore
echo '*.db' >> .gitignore
touch PROPOSAL.md && code PROPOSAL.md
```

Line-by-line explanation of the setup: identical to every prior module’s venv ritual, with one new addition worth flagging explicitly: `echo '*.db' >> .gitignore` — say why: SQLite database **files** (which Part 2–3 create) are generated data, not source code — regenerable from `schema.sql` plus whatever data gets inserted, and often not something you want tracked in version control (especially once real or realistic-looking data populates them) — the same underlying principle as excluding `.venv/`, applied to a new kind of generated artifact.

Have students fill in every field:

```

# ISM3232 Capstone Project Proposal
# Author: [Your Name]

## Project Name
[Give your system a clear, descriptive name]

## Business Problem
[What problem does this solve? Who has this problem? 2-3 sentences.]

## Primary User
[Who will use this system? Be specific.]

## Records Stored
[What data is stored? Name every type of record.]

## Business Rules
[At least 2 explicit rules. e.g., 'Requests over $1000 require manager approval']
1.
2.

## OOP Classes
[List each class with key attributes and methods]
- Class 1: [name] | attributes: [...] | methods: [...]
- Class 2: [name] | attributes: [...] | methods: [...]

## SQL Tables
[List each table with columns and types]
- Table 1: [name] | columns: [...]

## Streamlit Features
1. Add a record
2. View all records
3. Filter records
4. Update status
5. Display a report

## GenAI Feature
[What will AI do? e.g., 'Summarise a request description in one sentence']

## Out of Scope
[What are you explicitly NOT building?]

```

Demonstrate your own worked example, live, continuing the Week 12 event-registration domain, as a model of specificity:

Project Name

GalaTrack – Event Registration Manager

Business Problem

A nonprofit's annual gala currently tracks registrations in a shared spreadsheet that multiple staff edit simultaneously, causing overwritten entries and no reliable way to see confirmed revenue at a glance.

Primary User

The nonprofit's events coordinator, who processes 150–300 registrations per event and needs an at-a-glance revenue and confirmation status view.

Records Stored

Events (name, date, capacity). Registrations (attendee, event, price, status).

Business Rules

1. A registration over \$500 is flagged as VIP and routed to a dedicated check-in line.
2. An event cannot accept new registrations once confirmed registrations reach its capacity.

OOP Classes

- Class 1: Registration | attributes: reg_id, attendee_name, event, price, status | methods: confirm(), cancel(), is_vip()
- Class 2: Event | attributes: event_id, name, capacity | methods: is_full(registrations)

SQL Tables

- Table 1: events | columns: event_id INTEGER PK, name TEXT NOT NULL, capacity INTEGER NOT NULL
- Table 2: registrations | columns: reg_id INTEGER PK, attendee_name TEXT NOT NULL, event_id INTEGER NOT NULL, price REAL NOT NULL, status TEXT DEFAULT 'Pending'

Streamlit Features

1. Add a new registration
2. View all registrations for an event
3. Filter registrations by status

4. Confirm or cancel a registration
5. Display a revenue/status summary report

GenAI Feature

Generate a one-sentence welcome email draft for a newly confirmed attendee, using their name and event details.

Out of Scope

Payment processing, waitlist management, multi-event discount codes.

Facilitation notes on running the proposal review — higher stakes than Week 12’s design gate, worth a correspondingly careful bar:

- **What “approved” means here, specifically:** every field genuinely specific (a named business problem with a named user type, not a generic description); at least 2 real business rules with actual thresholds or conditions; OOP classes that map sensibly onto the business rules described; SQL tables whose columns genuinely support the records described; exactly 5 Streamlit features (the template’s five are a reasonable, achievable default — encourage most students to keep them rather than inventing a longer list); a GenAI feature that’s genuinely achievable in the time remaining (a one-sentence summarization or generation task, not an ambitious open-ended AI agent).
- **The most common reason to send a proposal back for revision:** scope that’s too large for four remaining weeks — three or more OOP classes doing genuinely different jobs, or business rules that imply significant additional infrastructure (user authentication, multi-tenant data separation, real payment processing). Redirect toward the *minimum* viable version of the idea, not the most ambitious one — say explicitly, this is a favor to the student’s own future weeks, not gatekeeping for its own sake.
- **A second common issue:** OOP classes and SQL tables that don’t actually correspond to each other — if the proposal’s “OOP Classes” section lists a class with attributes that don’t appear anywhere in the “SQL Tables” section (or vice versa), that’s worth flagging now, since Week 14 explicitly requires wiring the two together, and a mismatch here becomes a real blocker then.

Common student mistakes to watch for:

- A “Business Problem” that describes the *system* rather than the *problem* (“I will build an app that tracks requests” instead of “the office currently loses track of which requests are pending because there’s no shared view”) — redirect toward describing the pain point a real person has today, before any software exists to fix it.
- Business rules that aren’t actually rules, just descriptions (“requests have a status” is a fact, not a rule; “requests over \$1000 require manager approval” is a rule, since it implies real conditional logic) — push for something with a genuine if-shaped consequence.
- Five Streamlit features padded with near-duplicates (“view all records” and “view records list”

as two separate items) — a quick read-through catches this; redirect toward five genuinely distinct capabilities.

6.3 Part 2 — SQL Fundamentals (0:26–0:42, 16 min)

Teaching goal: The core SQL vocabulary — CREATE TABLE, INSERT, SELECT, WHERE, UPDATE ... SET, GROUP BY with aggregates — in the sqlite3 interactive shell, on data that directly parallels Weeks 10–11’s BusinessRequest system.

Say to the class:

“This should feel familiar in a specific way: every operation we run today maps directly onto something you already built in Python over the last three weeks. CREATE TABLE is class. INSERT is creating an instance. SELECT ... WHERE is a list comprehension filter. GROUP BY with SUM is your manager’s total_amount(). You already know these ideas — today is new vocabulary for concepts you’ve already built by hand.”

Live-code this in the terminal:

```
sqlite3 lab13.db
```

Then, inside the sqlite3 shell, run each statement:

```
-- Create a table
CREATE TABLE requests (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    requester  TEXT      NOT NULL,
    category   TEXT      NOT NULL,
    amount     REAL,
    status     TEXT      DEFAULT 'Pending'
);
```

Line-by-line explanation:

- CREATE TABLE requests (...) — say explicitly, direct callback to Week 11 Part 3’s mapping table: this line **is** the SQL equivalent of class BusinessRequest: — a fixed blueprint for what every row will contain.
- id INTEGER PRIMARY KEY AUTOINCREMENT — INTEGER is the column’s type; PRIMARY KEY marks this column as the unique identifier for every row (no two rows can share an id); AUTOINCREMENT means SQLite assigns the next number automatically — you never manually specify id when inserting, exactly as this lab’s upcoming INSERT statements demonstrate.
- requester TEXT NOT NULL — NOT NULL is a **constraint**: SQLite will refuse to insert a row missing this value, rather than silently allowing an incomplete record. Say explicitly: this is SQL’s version of a required field — Python’s equivalent would be a parameter with no default value in __init__.
- amount REAL — **no NOT NULL here, deliberately** — worth noting this specific column is allowed to be empty (a NULL value), unlike requester/category.
- status TEXT DEFAULT 'Pending' — direct SQL equivalent of Week 10’s self.status = 'Pending' in __init__ — a value automatically applied when a row is inserted without

explicitly specifying status, exactly parallel to Python's default-attribute pattern.

Insert three records:

```
INSERT INTO requests (requester, category, amount) VALUES ('Taylor', 'Travel', 1200);
INSERT INTO requests (requester, category, amount) VALUES ('Jordan', 'Equipment', 450);
INSERT INTO requests (requester, category, amount) VALUES ('Morgan', 'Software', 3500);
```

Line-by-line explanation: INSERT INTO requests (requester, category, amount) VALUES (...) — say explicitly, this is the SQL equivalent of BusinessRequest(101, 'Taylor', 'Travel', 1200) from Week 10 — creating one new record. Note that id and status are both omitted from the column list — id because AUTOINCREMENT assigns it automatically, status because its DEFAULT 'Pending' fills in automatically when not specified — worth pointing out this mirrors Python's __init__ not requiring status as a parameter at all, since it was always hardcoded to default there too.

Query all records:

```
.mode column
.headers on
SELECT * FROM requests;
```

Line-by-line explanation: .mode column and .headers on are **shell-specific formatting commands** (note: no trailing semicolon, and they start with a dot) — say explicitly, these aren't SQL itself, they're instructions to the sqlite3 program about how to *display* query results readably, worth running once at the start of any session. SELECT * FROM requests; — * means "every column"; this returns all three inserted rows.

Verified output:

id	requester	category	amount	status
1	Taylor	Travel	1200.0	Pending
2	Jordan	Equipment	450.0	Pending
3	Morgan	Software	3500.0	Pending

Filter:

```
SELECT * FROM requests WHERE amount > 1000;
```

Line-by-line explanation: direct SQL equivalent of [r for r in requests if r.amount > 1000] — say explicitly, this is precisely Week 11's mapping table row, now actually executed. **Verified output:** two rows (Taylor's \$1,200 and Morgan's \$3,500), Jordan's \$450 correctly excluded.

Update:

```
UPDATE requests SET status = 'Approved' WHERE id = 1;
SELECT * FROM requests;
```

Line-by-line explanation: UPDATE requests SET status = 'Approved' WHERE id = 1 — direct SQL equivalent of req_101.approve() — say explicitly: **WHERE id = 1 is essential here** — without it, UPDATE requests SET status = 'Approved' would set *every single row's* status to 'Approved' at once, a genuinely dangerous mistake worth flagging explicitly, parallel in spirit to Week 2's rm safety ritual (know exactly what you're about to affect before running a command that changes or destroys data).

Verified output (only id = 1's status changed):

id	requester	category	amount	status
1	Taylor	Travel	1200.0	Approved
2	Jordan	Equipment	450.0	Pending
3	Morgan	Software	3500.0	Pending

Aggregate report:

```
SELECT status, COUNT(*) as count, SUM(amount) as total FROM requests GROUP BY status;
```

Line-by-line explanation: this is the single most important line of Part 2 — the direct SQL equivalent of everything Week 11's RequestManager.summary_report() computed by hand, in Python, across multiple methods. GROUP BY status buckets rows by their status value (exactly like Week 6's .get(key, 0) accumulate-into-a-dict pattern, or Week 11's list_pending()/get_by_status()); COUNT(*) as count counts rows in each bucket; SUM(amount) as total sums the amount column within each bucket. Say explicitly: **one line of SQL is replacing what took an entire manager class's worth of Python methods to build by hand** — worth letting that land as a genuinely striking comparison, not glossing past it.

Verified output:

status	count	total
Approved	1	1200.0
Pending	2	3950.0

Exit the shell:

```
.quit
```

Common student mistakes to watch for:

- Forgetting the semicolon at the end of a SQL statement — the shell will simply wait for more input, showing a ...> continuation prompt instead of running the statement; a good, low-

stakes thing to demonstrate live so students recognize the “stuck” prompt and know to add the missing ;.

- Running UPDATE without a WHERE clause, as flagged above — worth demonstrating the consequence live, deliberately, on this throwaway lab13.db (not the capstone’s real database): run UPDATE requests SET status = 'Approved'; with no WHERE, then SELECT * FROM requests; to show all three rows now incorrectly 'Approved' — a memorable, safe-to-witness mistake.
- Confusing SQL’s = (used for both assignment in SET and comparison in WHERE) with Python’s == distinction — worth flagging explicitly: **SQL uses a single = for equality comparison in a WHERE clause**, unlike Python’s == — a genuinely different convention between the two languages, worth stating outright rather than letting students assume Python’s rules carry over.

Check for understanding: “Which Python method from Week 11’s RequestManager does GROUP BY status most directly replace?” (summary_report() — specifically the parts computing counts and totals per status; a good direct check that the mapping from Week 11 genuinely transferred to real, executable SQL today.)

6.4 Part 3 — Design Your Capstone Schema (0:42–0:56, 14 min)

Teaching goal: Write `schema.sql` for the student’s own approved capstone proposal, and test it directly in the `sqlite3` shell.

Say to the class:

“Now your own tables, from your own proposal’s ‘SQL Tables’ section. Real types, NOT NULL where it matters, a status column with a default — exactly the pattern from Part 2, applied to your own domain.”

Do:

```
touch schema.sql && code schema.sql
```

Demonstrate your own worked example’s schema, as a model — continuing the GalaTrack proposal:

```
-- schema.sql
CREATE TABLE events (
    event_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name     TEXT     NOT NULL,
    capacity INTEGER NOT NULL
);

CREATE TABLE registrations (
    reg_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    attendee_name   TEXT     NOT NULL,
    event_id        INTEGER NOT NULL,
    price           REAL     NOT NULL,
    status          TEXT     DEFAULT 'Pending',
    FOREIGN KEY (event_id) REFERENCES events(event_id)
);
```

Line-by-line explanation of the one genuinely new element — the FOREIGN KEY:

- `FOREIGN KEY (event_id) REFERENCES events(event_id)` — say explicitly, this is SQL’s mechanism for expressing exactly the relationship Week 11’s `Registration.event` attribute expressed in Python (one `Registration` object holding a reference to a specific `Event` object). In SQL, a table can’t hold a whole other row *inside* a cell the way a Python attribute can hold a whole other object — instead, `registrations.event_id` holds just the *id* of the related `events` row, and `FOREIGN KEY` formally declares that relationship, letting the database enforce it (refusing to insert a registration whose `event_id` doesn’t correspond to a real event, if foreign key enforcement is turned on — worth mentioning this enforcement isn’t automatic in SQLite by default, a detail beyond today’s required depth but worth a one-sentence flag if a curious student asks why an invalid `event_id` doesn’t immediately error).

Test it in the sqlite3 shell:

```
sqlite3 capstone.db
.read schema.sql
.schema
INSERT INTO events (name, capacity) VALUES ('Annual Gala', 200);
INSERT INTO registrations (attendee_name, event_id, price) VALUES ('Taylor', 1, 750);
SELECT * FROM registrations;
.quit
```

Line-by-line explanation:

- `.read schema.sql` — a shell command (dot-prefixed again) that runs every SQL statement in the named file, exactly as if each had been typed individually — say explicitly, this is the SQL equivalent of Python's `import`: pulling in and executing a whole file's worth of definitions in one step.
- `.schema` — another shell command, printing the `CREATE TABLE` statement(s) currently defined in this database, as SQLite itself understands them — a good verification that `.read` worked correctly, independent of trusting the source file was typo-free.
- The two `INSERT` statements and final `SELECT` — a genuine end-to-end test: create one event, one registration referencing it by `event_id`, and confirm the registration reads back correctly.

Verified output (`.schema`, then the final `SELECT`):

```
CREATE TABLE events (
  event_id INTEGER PRIMARY KEY AUTOINCREMENT,
  name     TEXT     NOT NULL,
  capacity INTEGER NOT NULL
);
CREATE TABLE registrations (
  reg_id      INTEGER PRIMARY KEY AUTOINCREMENT,
  attendee_name TEXT     NOT NULL,
  event_id    INTEGER NOT NULL,
  price       REAL      NOT NULL,
  status      TEXT      DEFAULT 'Pending',
  FOREIGN KEY (event_id) REFERENCES events(event_id)
);
reg_id  attendee_name  event_id  price  status
-----
1       Taylor         1         750.0  Pending
```

Common student mistakes to watch for:

- Using a type name that doesn't exist in SQLite (e.g., `VARCHAR(50)`, common in other database systems) — SQLite is famously permissive about types (it will often accept this without error,

due to “type affinity” rather than strict typing) but it’s worth steering students toward SQLite’s own core type names (TEXT, INTEGER, REAL) for clarity and consistency with what Part 2 demonstrated.

- Forgetting NOT NULL on columns that clearly should always have a value (an attendee’s name, an event’s name) — walk each student’s schema checking this against their own proposal’s described records.
- A FOREIGN KEY line referencing a table or column name that doesn’t actually match the other CREATE TABLE statement exactly (a typo) — SQLite may not immediately error on this depending on settings, so a careful read-through, not just a successful .read, is the real check.

Check for understanding: “Why does registrations store event_id (a number) rather than the event’s actual name?” (Storing just the ID and looking up the name via the relationship avoids duplicating the event’s name in every single registration row — if the event were ever renamed, only one row in events needs updating, not every related registration. This is a genuine, important database design principle — avoiding redundant, duplicated data — worth stating explicitly even at this introductory level.)

6.5 Part 4 — sqlite3 Shell Reference (0:56–1:04, 8 min)

Teaching goal: A README cheatsheet documenting every required SQL operation with a real example — the same “document what you learned” habit from Week 2’s command-reference README, now applied to SQL.

Say to the class:

“Same habit as Week 2’s command reference — but SQL this time. One example of each required operation, from your own actual work today, not copied generically.”

Do, live:

touch README.md && code README.md

Add a SQL reference section, with each line filled in using a real example from today’s own work:

SQL Reference

```
CREATE TABLE    -- define a table
  e.g. CREATE TABLE requests (id INTEGER PRIMARY KEY, ...);

INSERT INTO      -- add a record
  e.g. INSERT INTO requests (requester, amount) VALUES ('Taylor', 1200);

SELECT *         -- retrieve all records
  e.g. SELECT * FROM requests;

WHERE            -- filter records
  e.g. SELECT * FROM requests WHERE amount > 1000;

UPDATE ... SET   -- change a value
  e.g. UPDATE requests SET status = 'Approved' WHERE id = 1;

ORDER BY        -- sort results
  e.g. SELECT * FROM requests ORDER BY amount DESC;

GROUP BY        -- aggregate by column
  e.g. SELECT status, COUNT(*) FROM requests GROUP BY status;

COUNT / SUM / AVG -- aggregate functions
  e.g. SELECT SUM(amount) FROM requests;
```

Point out explicitly: ORDER BY and AVG weren’t run live in Part 2 — students should test these

themselves in the shell before documenting them, not just copy the template's placeholder syntax. This is worth stating directly: **a README documenting a command that was never actually run and verified is exactly the kind of unreliable documentation this course's habits are meant to prevent.**

Common student mistakes to watch for:

- Copying example syntax without actually running it first — walk the room checking that ORDER BY/AVG examples specifically were tested in the shell, since they weren't part of Part 2's guided sequence.

Check for understanding: “Which of these eight operations maps most directly onto something you've already built in RequestManager, and which feels most genuinely new?” (A good closing reflection — most students should name GROUP BY/aggregates as the most direct Python parallel and FOREIGN KEY-style relationships or ORDER BY as comparatively newer territory, though answers will reasonably vary.)

6.6 Part 5 — Ritual and Push (1:04–1:10, 6 min)

Teaching goal: Commit and push the proposal, schema, and README — note this week’s ritual is shorter than usual, since there’s no Python code yet to format, lint, or test.

Say to the class:

“No ruff or pytest today — there’s no Python code yet, just the proposal, schema, and README. Stage, commit, push.”

Live-code this:

```
git add . && git commit -m 'lab 13: proposal and SQL schema' && git push
```

Common student mistakes to watch for:

- Accidentally committing a `.db` file despite the `.gitignore` entry added in Part 1 — if `lab13.db` or `capstone.db` was created *before* `.gitignore` was updated to include `*.db`, Git may have already started tracking it; a quick `git status` before `add/commit` catches this, and if needed, `git rm --cached *.db` removes it from tracking without deleting the actual file.

Check for understanding: “If this week’s ritual has no `pytest` step, does that mean nothing about today’s work can be verified as correct?” (No — Part 2 and Part 3’s `SELECT` queries, run and read carefully in the shell, *are* the verification this week, just manual rather than automated; Week 14 is where automated `pytest` coverage returns, now including database-backed tests.)

7 Wrap-Up (last ~5 minutes)

Review the submission checklist together:

- ☐ PROPOSAL.md — all 10 fields completed, genuinely specific, **instructor-approved**
- ☐ schema.sql — CREATE TABLE statement(s) for the capstone, tested in the sqlite3 shell
- ☐ README.md — SQL reference section, all eight operations documented with real, tested examples
- ☐ Git commit made, with a message including “lab 13”
- ☐ Repo pushed
- ☐ GitHub repository URL pasted into Canvas

Preview Week 14: “Next week, Python and SQL finally connect — reading from and writing to your actual database from inside a Python script, using the schema you designed today. This is where your OOP classes from Weeks 10–12 and your SQL tables from today become one working system.”

8 Appendix A — Full Worked Example (PROPOSAL.md + schema.sql + SQL Reference)

The GalaTrack event-registration proposal used throughout this guide, complete and internally consistent — OOP classes matching SQL tables matching business rules. Use this to calibrate what an approved proposal looks like; do not distribute as a copyable answer.

```
# ISM3232 Capstone Project Proposal
# Author: [Instructor demo]

## Project Name
GalaTrack – Event Registration Manager

## Business Problem
A nonprofit's annual gala currently tracks registrations in a shared
spreadsheet that multiple staff edit simultaneously, causing overwritten
entries and no reliable way to see confirmed revenue at a glance.

## Primary User
The nonprofit's events coordinator, who processes 150–300 registrations
per event and needs an at-a-glance revenue and confirmation status view.

## Records Stored
Events (name, date, capacity). Registrations (attendee, event, price,
status).
```

Business Rules

1. A registration over \$500 is flagged as VIP and routed to a dedicated check-in line.
2. An event cannot accept new registrations once confirmed registrations reach its capacity.

OOP Classes

- Class 1: Registration | attributes: reg_id, attendee_name, event, price, status | methods: confirm(), cancel(), is_vip()
- Class 2: Event | attributes: event_id, name, capacity | methods: is_full(registrations)

SQL Tables

- Table 1: events | columns: event_id INTEGER PK, name TEXT NOT NULL, capacity INTEGER NOT NULL
- Table 2: registrations | columns: reg_id INTEGER PK, attendee_name TEXT NOT NULL, event_id INTEGER NOT NULL, price REAL NOT NULL, status TEXT DEFAULT 'Pending'

Streamlit Features

1. Add a new registration
2. View all registrations for an event
3. Filter registrations by status
4. Confirm or cancel a registration
5. Display a revenue/status summary report

GenAI Feature

Generate a one-sentence welcome email draft for a newly confirmed attendee, using their name and event details.

Out of Scope

Payment processing, waitlist management, multi-event discount codes.

-- *schema.sql*

```
CREATE TABLE events (  
    event_id  INTEGER PRIMARY KEY AUTOINCREMENT,  
    name      TEXT      NOT NULL,  
    capacity  INTEGER NOT NULL  
);
```

```
CREATE TABLE registrations (  
    reg_id    INTEGER PRIMARY KEY AUTOINCREMENT,  
    attendee_name TEXT NOT NULL,  
    event_id  INTEGER NOT NULL,  
    price     REAL NOT NULL,  
    status    TEXT DEFAULT 'Pending'
```

```

    reg_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    attendee_name   TEXT    NOT NULL,
    event_id        INTEGER NOT NULL,
    price           REAL    NOT NULL,
    status          TEXT    DEFAULT 'Pending',
    FOREIGN KEY (event_id) REFERENCES events(event_id)
);

```

Full verified Part 2 SQL sequence (on the generic requests practice table):

```

CREATE TABLE requests (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    requester   TEXT    NOT NULL,
    category    TEXT    NOT NULL,
    amount      REAL,
    status      TEXT    DEFAULT 'Pending'
);

INSERT INTO requests (requester, category, amount) VALUES ('Taylor', 'Travel', 1200);
INSERT INTO requests (requester, category, amount) VALUES ('Jordan', 'Equipment', 450);
INSERT INTO requests (requester, category, amount) VALUES ('Morgan', 'Software', 3500);

.mode column
.headers on
SELECT * FROM requests;
SELECT * FROM requests WHERE amount > 1000;
UPDATE requests SET status = 'Approved' WHERE id = 1;
SELECT * FROM requests;
SELECT status, COUNT(*) as count, SUM(amount) as total FROM requests GROUP BY status;

```

9 Appendix B — Extra Practice (only if the class finishes early)

If a student's proposal is approved and their schema tests cleanly well before time is up:

Extra — an ORDER BY and AVG round, genuinely run. Have students run `SELECT * FROM requests ORDER BY amount DESC;` and `SELECT AVG(amount) FROM requests;` in the shell themselves (both are part of Part 4's required documentation, but weren't part of Part 2's guided walkthrough) — confirming they've actually executed, not just copied, every operation they document.

Extra — a second table with a genuine WHERE + JOIN preview. For students whose capstone has two related tables (most will), have them try: `SELECT registrations.attendee_name, events.name FROM registrations JOIN events ON registrations.event_id = events.event_id;` — a JOIN is beyond this lab's required scope, but a working, verified

example (confirmed: returns Taylor | Annual Gala for this guide's sample data) is a genuinely motivating preview of Week 14's deeper Python-SQL integration work.